

Poly Money Maker: technical design of the live 15m mint system

Captured 19 September 2026 · 15m atomic mint · VM source of truth

This document explains the system as it runs on **19 September 2026**. It follows `mintbot.py`, the live atomic-mint trader, from discovering the next BTC 15m Up/Down window through a confirmed complete-set mint, optional loser/winner/held-leg sells, and the local state that survives a restart. Python examples are taken from the live VM tree `/home/ntemusejoel/poly-money-maker`. Hypothetical trades illustrate arithmetic; they are not performance claims.

Three documents have different jobs:

Document	Question it answers
<code>CURRENT.md</code>	What is running today, with which settings?
<code>AGENTS.md</code>	What must a coding agent know before changing anything?
<code>TECHNICAL_DESIGN.md</code>	How is the system built, and why do its money paths work this way?

This file replaces the earlier guided tour of `buybothourly.py` (hourly FAK entry / hedge / TP). That strategy is **retired**. Buybots, complement, DangerZone, shadow bots, and hourly-dense pathlog stay off. Do not start them from this document.

Read Parts I and II straight through. Part III walks mint and sell. Part IV covers helpers. Part V covers operations and sharp edges.

Contents

- Part I — Picture
 - The opportunity and its limits
 - Processes, wallet identities and files
 - Repository map and reading order
- Part II — Ideas the code assumes
 - Complete sets, CTF split, and why mint $\neq$ buy
 - Books, FAK, sized bids, and three prices
 - Python shape: values, state and side effects
 - External systems
 - JSON as durable memory
- Part III — Walking `mintbot.py`
 - Startup, lock, strategy load
 - The cycle: sells first, then mint
 - Eligibility: not-yet-open 15m windows
 - Capacity: `max_open_sets=1` and adjacent lookahead

- already_minted includes failed
- Relay submit: approve + split as one PROXY batch
- Reconcile: relay state → inventory confirm
- Sell path overview
- Loser dump: 3¢ → 2¢ after opposite ≥ 90¢
- Winner cash-out: prefer 0.999 / redeem; 0.99 only after cheap loser
- Held-leg dump: under 80¢ for 5s after loser sold
- Live-bid FAK vs fixed-limit FAK
- Hypothetical lifecycle: \$5 mint, loser @2¢, redeem winner
- Hypothetical lifecycle: held dump after a flip
- What this code does not prove
- Part IV — The buy/ helpers and pathlog
- Ownership map
- buy/mint_sell.py policy helpers
- buy/market.py, book.py, chain.py, contracts.py
- pathlog.py: public book recorder
- Part V — Operations, verification and sharp edges
- systemd units
- Live knobs (19 Sep 2026)
- Deploy boundary (VM is source of truth)
- Testing without constructing a live bot
- Landmines
- Glossary
- Source snapshot
- Part VI — Sequences & redeem
- End-to-end mint sequence
- Sell-side sequence (loser → winner/dump)
- Redeem path (what exists vs what does not)
- State after expiry

Part I — Picture

The opportunity and its limits

Polymarket's **BTC Up or Down 15m** markets are binary windows. Each window asks whether BTC finishes the quarter-hour up or down relative to that window's open. Outcome tokens pay **\$1** of collateral if that side wins and **\$0** if it loses.

A **complete set** is one Up share plus one Down share for the same condition. On-chain Conditional Token Framework (CTF) mechanics let you **split** \$N of collateral into N Up + N Down. That is a **mint**, not a directional buy. After the split you hold both legs. Economic edge then comes from:

1. Selling the **loser** cheaply once the book clearly prices it near zero (and the opposite near one), and

2. Preferring to **redeem** the winner at \$1 after resolution (or, only in narrow cases, cashing the winner out on the CLOB near \$1).

With a \$5 trial (shares=5), you pay about \$5 to mint 5 Up + 5 Down. If you sell the loser for \$0.10 total (5 × 2¢) and redeem the winner for \$5, gross is about \$5.10 before fees/gas abstraction — a thin edge that only works if loser fills are reliable and winner redeem is not accidentally sold too early at a worse price.

What this bot is not: it is not the old hourly FAK entry bot. It does not chase 90–95¢ asks on one side with an oracle. It does not “hedge” by buying the opposite leg after entry. The post-loser exit under 80¢ is deliberately named a **held dump** / **sell-side pass**, not a hedge.

Risk concentration: max_open_sets=1 means at most one full unsold bag blocks capacity (with a special adjacent-window exception described below). A failed relayer mint that used to remind-loop is treated as already attempted. A single toxic loser fill or a missed dump still matters at small size; scaling share count scales both edge and left-tail together.

Processes, wallet identities and files

Live tree: /home/ntemusejoel/poly-money-maker on Google Cloud VM poly-vm.

Unit	Program	Role	Observed policy
polymintbot.service	mintbot.py + gitignored strategy_mint.json	Atomic mint + optional sells	active / enabled
polypathlog.service	pathlog.py	Public CLOB path recorder (no orders)	active / enabled
Retired buy / danger / dense pathlog units	—	—	stopped / must stay off

Wallet roles (do not put secrets in this doc):

- **EOA / signer** — PRIVATE_KEY signs relayer PROXY batches and CLOB sells.
- **Funder / proxy** — FUNDER_ADDRESS is the Polymarket proxy that holds pUSD and outcome tokens (live watch address historically 0x8222...).
- **Relayer auth** — API key headers must match the signer address.

Durable local files (gitignored where noted):

Path	Purpose
strategy_mint.json	Live knobs (gitignored)
strategy_mint.example.json	Committed template
positions_mint.json	Intent state machine + sell flags
.heartbeat_mint / .heartbeat_pathlog	Liveness stamps
.mintbot.lock / .pathlog.lock	Single-instance flock
mintbot.log / pathlog.log	Append logs
.env	Secrets — never read into chat or commit

Repository map and reading order

```

mintbot.py           # live entry: discover → mint → reconcile → sell
pathlog.py           # 15m-only book recorder
strategy_mint.json    # LIVE knobs (gitignored)
strategy_mint.example.json
positions_mint.json    # LIVE state (gitignored)
buy/
  mint_sell.py        # pure sell policy (persist, classify, ladders)
  market.py           # Gamma/CLOB discovery → MintMarket
  book.py              # sized top-of-book
  chain.py             # eth_call balances / prechecks
  contracts.py         # approve + split calldata
deploy/
  polymintbot.service
  polypathlog.service
tests/                # unit tests; do not import mintbot.py wholesale
CURRENT.md / AGENTS.md / TECHNICAL_DESIGN.md

```

Reading order for a new engineer: this file Parts I–II, then `buy/mint_sell.py`, then `manage_sells` and `run_cycle` in `mintbot.py`, then `CURRENT.md` for today’s knobs.

Part II — Ideas the code assumes

Complete sets, CTF split, and why mint ≠ buy

On Polymarket (Polygon), collateral (pUSD) can be split through the CTF / adapter into a pair of outcome ERC-1155 positions for a `condition_id`. **Minting** creates both legs atomically relative to your inventory: after confirmation you should observe roughly `before + shares` on Up and Down.

Buying one outcome on the CLOB is a different trade: you pay the ask for one token and never receive the other. The mint bot’s edge thesis is “pay ~\$1 for the pair, sell trash, redeem (or carefully cash) the rest,” not “pick a side at 97¢.”

Gas and batching are abstracted by Polymarket’s **relayer**: the bot builds a PROXY transaction that typically includes allowance/approve plus split, submits it once, then polls relayer state until inventory shows up on-chain.

Books, FAK, sized bids, and three prices

The CLOB is a central limit order book. A **bid** is what buyers will pay; an **ask** is what sellers want.

A **sized bid** in this codebase is the best bid that still has at least `sell_min_bid_size` (live 1.0) size. Thin one-lot prints are ignored so a 1-share tease at 2¢ does not arm a 5-share dump.

FAK (Fill And Kill) / marketable limit sell: take liquidity down to your limit; cancel the rest. The bot uses FAK for loser ladders and for winner/held dumps.

Three prices that must not be conflated:

1. **Arm threshold** — e.g. loser $\leq 3\text{¢}$ with opposite $\geq 90\text{¢}$, or held bid $< 80\text{¢}$.
2. **Limit posted** — what the order is allowed to cross (ladder $3\text{¢} \rightarrow 2\text{¢}$, or live bid for winner/dump).
3. **Average fill** — what actually cleared (may be better than limit).

Live-bid FAK: once a winner/dump path is allowed to fire, the limit is the current sized bid (e.g. 0.99), not a stale fixed 0.999 that Polymarket rejects when the book max is 0.99.

Python shape: values, state and side effects

`buy/mint_sell.py` is intentionally pure-ish: given bids and knobs, return legs, persist decisions, ladder limits. No network.

`mintbot.py` owns side effects: HTTP to Gamma/CLOB/relayer, `eth_calls`, file IO, notifications, systemd process lifetime.

`positions_mint.json` is the durable state machine. Restart must not remind a market already `failed` / `confirmed` / `completed`, and must not forget `sold_loser` / `sold_leg`.

External systems

System	Use
Gamma API	Discover 15m markets / tokens / times
CLOB API	Sized bids; FAK sells
Data API	Optional position checks
Relayer v2	Submit PROXY mint batch; poll <code>STATE_*</code>
Polygon RPC	CTF balances / inventory confirm
ntfy (optional)	Operator push on mint/sell

JSON as durable memory

Top-level `positions_mint.json` shape:

```
{
  "daily": { "2026-09-19": 15.0 },
  "intents": {
    "<condition_id>": {
      "status": "confirmed",
      "slug": "btc-updown-15m-...",
      "shares": 5.0,
      "start_ts": 1789798500.0,
      "end_ts": 1789799400.0,
      "up_token": "...",
      "dn_token": "...",
      "transaction_id": "...",
      "relayer_state": "STATE_CONFIRMED",
      "sold_loser": true,
      "sold_leg": "up",
      "sell_limit": 0.03,
      "sold_winner": false,
      "sold_dump": false,
      "sell_loser_armed_at": 0.0,
      "sell_winner_armed_at": 0.0,
      "sell_dump_armed_at": null
    }
  }
}
```

Statuses move roughly: submitting → pending/executed/mined → confirmed_waiting_inventory → confirmed → (sells) → completed, or failed on relayer failure.

Part III — Walking mintbot.py

Startup, lock, strategy load

`main()` installs signal handlers, acquires `.mintbot.lock`, loads `strategy_mint.json` merged over `DEFAULTS`, validates knobs, constructs market gateway + chain reader, then loops: `reload_strategy` → `run_cycle` → `sleep poll_s` (live 5s).

`dry_run=true` must not submit mints or live sells. Live VM has `dry_run=false`, `entry_enabled=true`, `sell_enabled=true`.

The cycle: sells first, then mint

Each `run_cycle`:

1. `manage_sells` (always attempted first if `sell_enabled`) — can free capacity by marking `sold_loser`.
2. Reconcile open relayer intents / inventory.
3. If entry disabled → return.
4. Discover series markets; filter eligible; skip `already_minted`; skip owned tokens.
5. Pick earliest eligible; if `mint_slots_full(..., pick.start_ts)` → capped.
6. Daily notional cap; balance precheck; build approve+split; `submit_mint_batch`; record intent.

Sells-before-mint matters: selling the loser can clear the `max_open_sets` blocker for the adjacent window.

Eligibility: not-yet-open 15m windows

`eligible_markets` keeps markets that:

- have **not** started (`start_ts > now`),
- open within `(enter_min_ttm_min, enter_max_ttm_min]` minutes (live 0–16),
- are active, not closed, not neg-risk,
- optionally `accepting_orders`.

The bot **never mints a live (already open) window**. That is why missing the adjacent lookahead used to skip an entire quarter-hour: by the time the prior bag expired, the next market was already open and ineligible.

Capacity: `max_open_sets=1` and adjacent lookahead

`open_intent_count` counts intents in `ACTIVE_STATUSES` whose market has not been expired for >120s, **excluding** intents with `sold_loser` / `sold_leg`. Winner-only redeem holds must not consume the mint slot.

`mint_slots_full(state, cfg, now, candidate_start_ts)`:

- If fewer than `max_open_sets` full bags → not full.
- Else allow **only** the adjacent next window: `soonest_full_end ≤ candidate_start < soonest_full_end + 900`.
- If we already hold that next window, or the candidate is further out → full.

Live: `max_open_sets=1`. Holding 1:30–1:45 still permits minting 1:45–2:00 beforehand; it does **not** permit minting 2:00–2:15 while 1:30 is still a full bag.

already_minted includes failed

```
status in ACTIVE_STATUSES u {completed, failed}
```

A relayer `STATE_FAILED` marks the intent `failed`. Without counting `failed`, the bot reminted the same `condition_id` in a hot loop (seen on the 2:15 window). There is **no** automatic retry/backoff yet — failed means skip that window.

Relayer submit: approve + split as one PROXY batch

`submit_mint_batch`:

1. Load `PRIVATE_KEY` / `FUNDER_ADDRESS`.
2. Fetch relay payload (nonce + relay address) for PROXY type.
3. Encode proxy calls (approve/allowance as needed + split).
4. Build signed proxy request; require derived `proxywallet == funder`.
5. POST `/submit` with relayer auth headers.
6. Return `transactionID` or error string.

On success the intent is stored with tokens, shares, `start_ts/end_ts`, and `transaction_id`.

Reconcile: relayer state → inventory confirm

While status is submitting/pending/executed/mined, poll relayer:

- `STATE_FAILED` / `STATE_INVALID` → `failed`
- `STATE_CONFIRMED` → `confirmed_waiting_inventory` (naming may vary slightly in logs)
- mined/executed intermediate states update accordingly

Then `eth_call` CTF balances. When Up and Down each reach `before + shares` within tolerance → `confirmed` and notify. If the market ended and balances are flat, mark `completed`.

Sell path overview

`manage_sells` runs only for intents still inside their window (`now ≤ end_ts`) and in confirmed-like statuses. For each intent it fetches sized Up/Down bids, then evaluates three exits in order:

1. **Winner cash-out** (rich bid path)
2. **Held-leg dump** (only after loser sold; poor bid path)
3. **Loser dump** (cheap bid + rich opposite)

A cooldown (`sell_cooldown_s`, live 3s) gates attempts after any sell try.

Loser dump: 3¢ → 2¢ after opposite ≥ 90¢

Policy (`classify_loser` / equivalent):

- Sized loser bid $\leq$ `sell_threshold` (0.03)
- Sized opposite bid $\geq$ `sell_opposite_min` (0.90)
- Not both cheap (ambiguous)
- Persist that condition for `sell_persist_s` (5s) via `persist_ready`
- Then FAK ladder: threshold → floor (3¢ → 2¢), sized to inventory latch

On full fill: set `sold_loser=true`, `sold_leg="up"|"dn"`, store `sell_limit` (fill/limit evidence). Inventory latch distinguishes “await mint settlement” zeros from true flat.

Winner cash-out: prefer 0.999 / redeem; 0.99 only after cheap loser

Default `sell_winner_min=0.999`. Unconditional 0.99 cash-out was rejected: it cuts margin versus redeeming at \$1.

Cheap-loser gate: if `sold_loser` and recorded loser price $\leq$ `sell_winner_cheap_if_loser_le` (0.03), then `effective_winner_min = min(0.999, sell_winner_min_cheap=0.99)`.

When the sized winner bid meets `effective_winner_min` for `sell_persist_s`, **live-bid FAK** the winner (limit = current sized bid). Mark `sold_winner`.

If the book never reaches 0.999 and the cheap gate is closed, the bot holds for redeem after expiry (sells stop at `end_ts`).

Held-leg dump: under 80¢ for 5s after loser sold

This is the sell-side pass added 19 Sep 2026. It is **not** a hedge.

Preconditions (all required):

- `sell_dump_enabled` (live true)
- `sold_loser` (or truthy `sold_leg`)
- not already `sold_dump` / `sold_winner`
- `sold_leg` is "up" or "dn" so the held leg is well-defined
- sized held bid is not `None` and `< sell_dump_below` (0.80)
- that condition persists `sell_dump_persist_s` (5.0)
- not in sell cooldown

Action: live-bid FAK the held token; on success set `sold_dump=true` and `sold_winner=true` (so winner cash-out will not double-sell).

Scope: only the remaining leg after a loser fill. Full sets with neither leg sold never arm dump. After `end_ts`, `manage_sells` skips the intent (same as other sells).

Live-bid FAK vs fixed-limit FAK

Path	Limit choice	Why
Loser	Ladder 0.03 → 0.02	Intentionally walk down to floor
Winner (allowed)	Current sized bid	Book often tops at 0.99; posting 0.999 is rejected
Held dump	Current sized bid	Same rejection class; dump fires precisely when bid is <i>weak</i>

Observed failure mode before the fix: winner armed at bid 0.99 but FAK posted 0.999 → `invalid price ... max: 0.99`.

Hypothetical lifecycle: \$5 mint, loser @2¢, redeem winner

1. T-12m: mint 5/5 for next window; intent `confirmed`.
2. Mid-window: Up sized bid 0.02, Down 0.97 for 5s → FAK sell Up @3¢ then 2¢; `sold_leg=up`, `sold_loser=true`, `sell_limit≈0.02`.
3. Down never reaches 0.999; cheap gate would allow 0.99 but bid stalls at 0.97 → no winner cash-out.
4. Held dump requires bid `<0.80`; 0.97 does not qualify.
5. After end: sell loop stops; redeem Down for ~\$5. Gross $\approx \$5 + \text{loser proceeds} - \text{fees}$.

Hypothetical lifecycle: held dump after a flip

1. Loser sold as above; holding Down.
2. Tape flips: Down sized bid falls to 0.74 and stays ≤ 5 s under 0.80.
3. Dump arms → live-bid FAK Down @~0.74; `sold_dump=true`.
4. Result is a realized loss versus redeem, accepted as left-tail control after the loser already paid a scrap.

What this code does not prove

- That 2¢ loser fills always exist when opposite is 90¢.

- That redeem will be claimed automatically (operator/process may still need a redeem path outside this doc's sell loop).
- That adjacent minting always beats skipping (relayer can still `STATE_FAILED`).
- That dump at 80¢ is optimal; it is an operator-chosen circuit breaker.

Part IV — The buy/ helpers and pathlog

Ownership map

Module	Owner of
<code>mintbot.py</code>	Process, knobs merge, relayer, CLOB sells, state file
<code>buy/mint_sell.py</code>	Pure sell policy
<code>buy/market.py</code>	Discovery / <code>MintMarket</code>
<code>buy/book.py</code>	Sized BBO parse
<code>buy/chain.py</code>	RPC reads
<code>buy/contracts.py</code>	Calldata for mint batch
<code>pathlog.py</code>	Separate process; read-only books

buy/mint_sell.py policy helpers

- `parse_sell_fill_shares` — share leg from CLOB response (not USDC `takingAmount`).
- `inventory_latch` — await vs already_flat vs has_inventory.
- `classify_loser` — which leg is loser / both_cheap / wick_unconfirmed.
- `persist_ready` — arm → waiting → ready over `persist_s` (resets when qualify drops).
- `winner_cashout_leg` — unique leg whose sized bid $\geq$ `winner_min`.
- `loser_ladder_limits` — [threshold, floor] style limits.

Defaults mirror mintbot sell knobs including dump keys.

buy/market.py, book.py, chain.py, contracts.py

Discovery builds `MintMarket` with `condition_id`, `up_token`, `dn_token`, `start_ts`, `end_ts`, `slug`, `flags`.

Book helper returns best bid with minimum size. Chain helper reads ERC-1155 positions. Contracts helper encodes the atomic mint path used by the relayer batch.

pathlog.py: public book recorder

Separate systemd unit. `SERIES = ["btc-up-or-down-15m"]` only. Polls CLOB books, appends JSONL ticks under `pathlog/`, prunes by age/size, optionally records resolution. **No orders**. Used for research/backtests (`check_path_backtest.py`), not for mint decisions today (mint sells are book-only, no pricing oracle).

Part V — Operations, verification and sharp edges

systemd units

`deploy/polymintbot.service` runs `.venv/bin/python mintbot.py` with `EnvironmentFile=.env`, `Restart=always`.

`deploy/polypathlog.service` runs `pathlog.py` (no env file required for public books).

Never enable retired buy units from memory of old docs.

Live knobs (19 Sep 2026)

From VM `strategy_mint.json`:

Knob	Live value	Meaning
<code>entry_enabled</code>	true	Allow new mints
<code>dry_run</code>	false	Real mint/sell
<code>shares</code>	5	Complete set size (\$5 trial)
<code>enter_max_ttm_min</code>	16	Mint when window opens within 16m
<code>series_slugs</code>	<code>[btc-up-or-down-15m]</code>	15m only
<code>max_open_sets</code>	1	Capacity (see adjacent rule)
<code>max_daily_notional</code>	100	Daily mint spend cap
<code>poll_s</code>	5	Cycle sleep
<code>sell_enabled</code>	true	Enable manage_sells
<code>sell_threshold / sell_floor</code>	0.03 / 0.02	Loser ladder
<code>sell_opposite_min</code>	0.90	Opposite must be rich
<code>sell_persist_s</code>	5	Loser/winner persist
<code>sell_cooldown_s</code>	3	Between attempts
<code>sell_winner_min</code>	0.999	Prefer redeem-quality bid
<code>sell_winner_cheap_if_loser_le</code>	0.03	Gate for 0.99 winner
<code>sell_winner_min_cheap</code>	0.99	Winner limit if gated
<code>sell_dump_enabled</code>	true	Held-leg dump on
<code>sell_dump_below</code>	0.80	Dump arm threshold
<code>sell_dump_persist_s</code>	5	Dump persist

Deploy boundary (VM is source of truth)

Operational rule: **VM files win**. GitHub is backup/history. Copy VM → GitHub; do not blindly merge GitHub onto the VM. Live `strategy_mint.json` and `positions_mint.json` stay gitignored. Example knobs are not authorization to trade.

After code pull: restart **only** `polymintbot` / `polypathlog` when the operator asks.

Testing without constructing a live bot

Do not import `mintbot.py` in unit tests (credentials, lock, clients). Test `buy/mint_sell.py` and AST-extracted pure functions. `python -m unittest` in a disposable sandbox.

Landmines

1. **Failed remind storm** — `failed` must remain in `already_minted`.
2. **Skipping the next window** — without adjacent lookahead, `max_open_sets=1` + “never mint open markets” skips a quarter-hour.
3. **Winner at 0.999 on a 0.99 book** — use live-bid FAK once allowed.
4. **Dump without `sold_leg`** — held leg cannot be inferred; loser path must set `sold_leg`.
5. **Sells stop at `end_ts`** — no dump/cash-out after expiry in `manage_sells`; redeem is the remaining path.
6. **Importing mintbot in tests** — can take the flock or load `.env`.
7. **Confusing mint with buybot docs** — old hourly TDD describes a different money path.

Glossary

Term	Meaning here
Complete set	1 Up + 1 Down for one condition
Mint / split	Collateral → both outcome tokens
Loser dump	Sell cheap leg after opposite is rich
Winner cash-out	Sell rich leg near \$1 (gated)
Held dump	After loser sold, sell held leg if weak (<80¢)
Redeem	Exchange winning tokens for \$1 collateral after resolution
FAK	Fill-and-kill marketable limit
Sized bid	Best bid with minimum size
Adjacent lookahead	Mint next 15m while still holding current full bag
Relayer PROXY	Polymarket-submitted batched tx from proxy wallet

Source snapshot

- Host: Google VM `poly-vm` (`/home/ntemusejoel/poly-money-maker`)
- Services: `polymintbot.service`, `polypathlog.service` active
- Primary sources: `mintbot.py` (~1426 lines), `buy/mint_sell.py` (~166), `pathlog.py` (~527)
- Strategy: `gitignored strategy_mint.json` as tabulated in §29
- Document date: 19 September 2026 (rev: sequences + redeem)
- Prior document replaced: hourly `buybothourly.py` guided tour (9 Sep 2026 era)

Appendix A — Cycle pseudocode (faithful to live control flow)

```

every poll_s seconds:
    cfg = load_strategy()                # merge strategy_mint.json over DEFAULTS
    manage_sells(cfg, state, chain)      # may set sold_loser / sold_winner / sold_dump
    reconcile_intents(...)               # relayer poll + inventory confirm

    if not cfg.entry_enabled: return "disabled"

    markets = gateway.discover(cfg.series_slugs)
    candidates = eligible_markets(markets, cfg, now)  # NOT YET OPEN, within TTM band
    pick = first candidate where:
        not already_minted(condition_id)            # includes failed
        and wallet does not already hold tokens
    if no pick: return "idle"

    if mint_slots_full(state, cfg, now, pick.start_ts): return "capped_open"
    if daily_spent + shares > max_daily_notional: return "capped_daily"

    precheck balances / contracts
    calls = build approve + split(shares)
    tx_id, err = submit_mint_batch(calls)
    if err: status=failed; return
    persist intent{status=pending, start_ts, end_ts, up/dn tokens, shares, tx_id}

```

Appendix B — Sell decision table

Precondition	Persist	Action	Flags set
Loser sized bid ≤ 0.03 AND opposite ≥ 0.90 AND not both cheap	5s	FAK ladder 0.03 $\rightarrow$ 0.02	<code>sold_loser</code> , <code>sold_leg</code>
Winner sized bid $\geq$ <code>effective_winner_min</code> (0.999, or 0.99 if loser ≤ 0.03)	5s	Live-bid FAK winner	<code>sold_winner</code>
<code>sold_loser</code> AND held sized bid < 0.80	5s	Live-bid FAK held	<code>sold_dump</code> , <code>sold_winner</code>
<code>now > end_ts</code>	—	No CLOB sells	(redeem outside this loop)
Within <code>sell_cooldown_s</code> of last attempt	—	Skip fire	—

`effective_winner_min` formula:

```

effective = sell_winner_min                # 0.999
if sold_loser and sell_limit <= sell_winner_cheap_if_loser_le:
    effective = min(effective, sell_winner_min_cheap) # 0.99

```

Appendix C — Intent status machine

```
submitting → pending → executed/mined → confirmed_waiting_inventory → confirmed
                                         ↳ completed (flat after end)
                               ↳ failed   (STATE_FAILED / STATE_INVALID; already_minted forever)
```

ACTIVE_STATUSES (count toward open bags unless loser sold / expired+120s): submitting, pending, executed, mined, confirmed_waiting_inventory, confirmed.

Appendix D — Why adjacent lookahead exists

Timeline bug without lookahead (max_open_sets=1):

1. Mint window A (1:30–1:45) at 1:20. Intent confirmed; slot full.
2. At 1:32, window B (1:45–2:00) is eligible on time, but slot full → skip.
3. At 1:45+ε, A expires; slot frees. But B has **started**, and eligibility forbids open markets → B never minted.
4. Bot jumps to C (2:00–2:15).

Fix: while holding a full bag ending at `end_ts`, allow minting the candidate whose `start_ts` is exactly that adjacent boundary. Still forbid a second lookahead.

Combined with `sold_loser` freeing the slot mid-window, the bot can mint the next set after the loser dump without waiting for expiry.

Appendix E — Economic sketch (not a promise)

Assume shares=5, lossless fees for arithmetic only.

Path	Cash out	Comment
Mint	−5.00	Split collateral
Loser @ 0.02	+0.10	5 × 2¢
Redeem winner	+5.00	Post-resolution
Net	+0.10	Thin; fees can erase it

Path	Cash out	Comment
Mint	−5.00	
Loser @ 0.02	+0.10	
Held dump @ 0.74	+3.70	Circuit breaker
Net	−1.20	Paid to cut left tail

Path	Cash out	Comment
Mint	-5.00	
Loser @ 0.03	+0.15	
Winner live FAK @ 0.99	+4.95	Only if cheap-loser gate open
Net	+0.10	Same ballpark as redeem; redeems avoid CLOB fee/slip

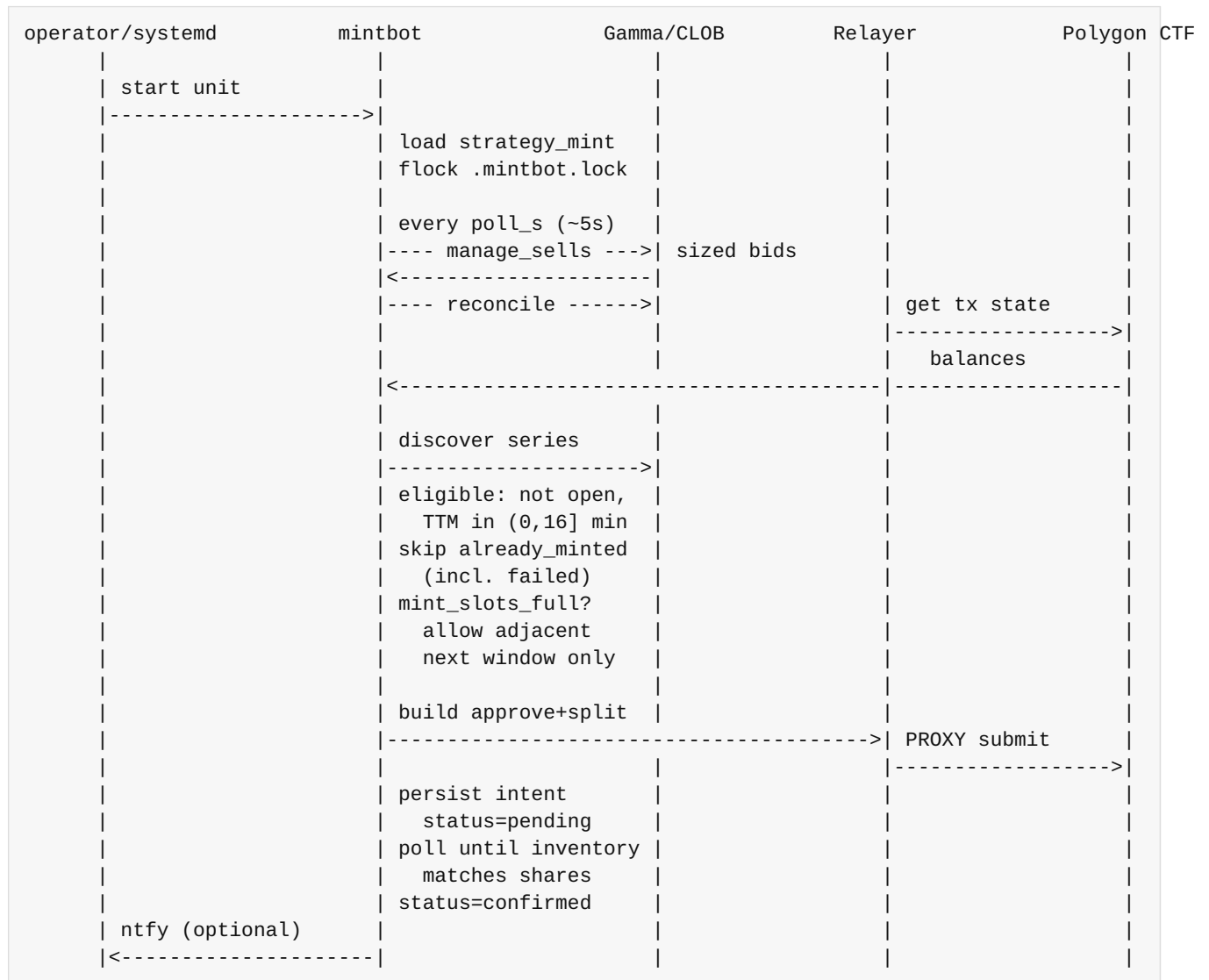
Appendix F — Operator checklist

1. `systemctl is-active polymintbot polypathlog` → both active.
2. `jq . strategy_mint.json` → confirm shares, `sell_`, `dump_` (never commit this file).
3. Tail `mintbot.log` for `mint_confirmed`, `sell_loser_done`, `sell_dump_done`, `mint_failed`.
4. After a `mint_failed`, expect **no** remind of that slug; wait for next window.
5. Code change on VM → restart **only** `polymintbot` when you ask.
6. GitHub sync is backup; VM remains SoT.

Part VI — Sequences and redeem

ASCII diagrams below are the PDF-safe form of sequence charts. They match the live `mintbot.py` control flow on 19 September 2026.

End-to-end mint sequence



Notes:

1. **Sells run before mint** each cycle. A mid-window loser fill can free `max_open_sets` so the adjacent mint is allowed sooner.
2. **Never mints an already-open window.** If adjacent lookahead fails, that quarter-hour is skipped forever for this bot.
3. Relayer `STATE_FAILED` → intent failed → `already_minted` stays true → no remint loop.

Sell-side sequence (loser → winner / held dump)

mintbot	CLOB book	inventory latch	flags on intent
for each confirmed			
intent with now<=end			
---- sized Up/Dn bids ->			
<-----			
[A] winner path			
effective_min = 0.999			
if sold_loser and			
sell_limit<=0.03:			
effective_min=0.99			
if sized winner bid			
>= effective_min			
for persist_s:			
---- live-bid FAK ----->			
		---- has shares? ----->	
			sold_winner
[B] held dump path			
requires sold_loser			
held = opposite of			
sold_leg			
if sized held bid			
< 0.80 for 5s:			
---- live-bid FAK ----->			
			sold_dump
			+ sold_winner
[C] loser path			
loser<=0.03 and			
opposite>=0.90 for 5s			
---- FAK 0.03 then 0.02>			
			sold_loser
			sold_leg=up dn
			sell_limit~fill

Ordering in code is **A** → **B** → **C** inside one intent iteration. That matters: a bag that already sold the loser can cash out or dump the held leg before another loser attempt (loser already flagged off).

Cooldown: after any sell attempt, `sell_cooldown_s` (3s) suppresses the next fire.

Redeem path — what exists vs what does not

What “redeem” means

After a 15m window resolves, the winning outcome token can be **redeemed** through the CTF for ~\$1 collateral per share. That is an on-chain call (or Polymarket UI / relayer helper), separate from CLOB sells.

Economically the mint thesis prefers:

1. Sell loser for scraps (2–3¢),
2. **Redeem** winner at \$1,

rather than selling the winner at 0.99 on the CLOB (which donates $\sim 1\text{¢} \times \text{shares}$ plus fees versus redeem).

What mintbot does today

Step	Implemented?	Where
Stop CLOB sells after <code>end_ts</code>	Yes	<code>manage_sells</code> skips intents with <code>now > end_ts</code>
Keep winner inventory unmarked if never cashed out	Yes	no forced sell at expiry
Free mint slot while winner sits for redeem	Yes	<code>sold_loser</code> excluded from open-slot count; also <code>expiry+120s</code>
Call CTF <code>redeemPositions</code> / merge automatically	No	not in <code>mintbot.py</code>
Relayer batch for redeem	No	mint-only submit path
Mark intent <code>redeemed</code> after payout	No	status may become <code>completed</code> when flat post-expiry

So “kept opposite for redeem” in logs is an **inventory policy**, not an automated redeem worker. Today, redemption is expected via:

- Polymarket portfolio UI / built-in redeem, or
- a future/manual script the operator runs,

not via the mint loop itself.

Why the design stops at “hold for redeem”

1. **Margin:** live-bid 0.99 cash-out was explicitly gated; unconditional 0.99 was rejected.
2. **Window boundary:** once `end_ts` passes, CLOB prices for that market become resolution-driven and the bot refuses further FAK risk.
3. **Scope control:** mint + sell-side pass is already enough surface area; auto-redeem adds another relayer/CTF path and failure mode (wrong condition index, partial redeem, gas/relayer auth).

Honest gap / future work

If you want the bot to close the cash loop without the UI, a follow-on would look like:

1. After `end_ts` + short delay, read resolution (Gamma or CTF payout vector),
2. If winner tokens remain and payout is live, submit redeem via relayer PROXY,
3. Confirm collateral increase; set `redeemed_at` / status `completed`,
4. Never redeem while CLOB dump/cash-out still eligible (`now <= end_ts`).

Until that exists, treat redeem as **operator-owned**, and treat mintbot as **mint + intra-window sell policy**.

State after expiry

```

now <= end_ts
└─ manage_sells active (loser / winner / dump)

now > end_ts
├─ manage_sells: skip this intent
├─ open_intent_count: still counts full bag until end_ts+120
│   └─ unless sold_loser already cleared the slot
├─ after end_ts+120: intent no longer blocks mint capacity
└─ if balances flat → reconcile may mark completed
    else tokens sit until human/UI/future redeem

```

Adjacent mint may already have been submitted **before** expiry (lookahead). That is intentional and is the main fix for the “skipped 15m” bug.

End of technical design.